

SPRINT 2

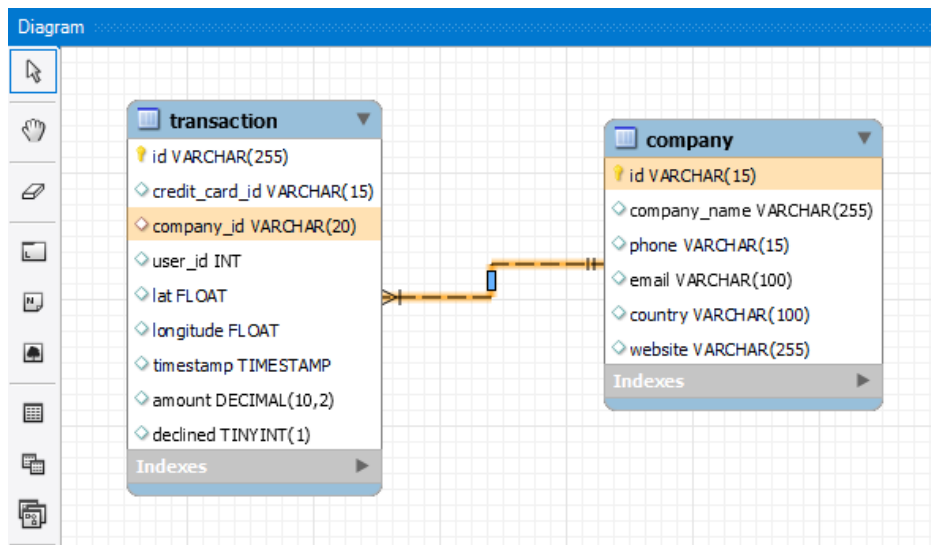
SQL

Exercici 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

La base de datos tiene dos tablas. La tabla “**company**” tiene una columna “*id*” como Primary Key, y el resto de columnas contienen la información de cada compañía, como el nombre (*company_name*), teléfono (*phone*), *email*, país (*country*), y página web (*website*), todos almacenados como VARCHARs, aunque de diferentes medidas acorde a su uso.

la tabla “**transactions**” tiene también una columna “*id*” como Primary Key, y luego columnas con información de la transacción, como “*credit_card_id*” (que hace referencia al id de la tabla credit_card; el identificador de la compañía (*company_id*), que es la Foreign Key que hace referencia al id de la tabla company; el identificador de usuario (*user_id*), que hace referencia al id de la tabla user; la latitud y longitud (*lat* y *longitude*) de donde se realizó la compra; el día y hora de la compra (*timestamp*); la cantidad pagada (*amount*); y si la compra ha sido rechazada o no (*declined*), que está en formato booleano.



El diagrama de la imagen muestra la relación entre las tablas, así como el tipo de dato que son cada una. Resaltado en amarillo-anaranjado se ve a través de qué variable se conectan las tablas: *company_id* hace referencia a *id* de la tabla **company**. La relación que se establece entre ambas es de **N:1**, es decir, varias transacciones pueden ser realizadas por una compañía, o una compañía puede realizar varias transacciones.

Exercici 2

Utilitzant JOIN realitzaràs les següents consultes:

- Llistat dels països que estan generant vendes.

The screenshot shows a SQL IDE window titled "Query 1" and "SQL File 4*". The query editor contains the following SQL code:

```
4 # Utilitzant JOIN realitzaràs les següents consultes:
5 # Llistat dels països que estan generant vendes.
6
7 • SELECT DISTINCT c.country
8   FROM transaction AS t
9   JOIN company AS c
10  ON t.company_id = c.id
11  WHERE t.declined = 0;
12
```

The "Result Grid" shows the following data:

country
Germany
Australia
United States
New Zealand
Norway

The "Output" section shows the "Action Output" table:

#	Time	Action	Message	Duration / Fetch
100113	12:34:43	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longit...	1 row(s) affected	0.000 sec
100114	12:38:26	SELECT DISTINCT c.country FROM transaction AS t JOIN company AS c O...	15 row(s) returned	0.000 sec / 0.000 sec

Seleccionamos los países sin repetir duplicados con un join entre **company** y **transaction**, con un filtro de *declined* = 0 para quedarnos con los registros de transacciones que han sido exitosas.

- Des de quants països es generen les vendes.

The screenshot shows the same SQL IDE window. The query editor contains the following SQL code:

```
12
13 # Des de quants països es generen les vendes.
14
15 • SELECT COUNT(DISTINCT c.country) AS total_paises
16   FROM transaction AS t
17   JOIN company AS c
18   ON t.company_id = c.id
19   WHERE t.declined = 0;
20
```

The "Result Grid" shows the following data:

total_paises
15

The "Output" section shows the "Action Output" table:

#	Time	Action	Message	Duration / Fetch
100114	12:38:26	SELECT DISTINCT c.country FROM transaction AS t JOIN company AS c O...	15 row(s) returned	0.000 sec / 0.000 sec
100115	12:42:24	SELECT COUNT(DISTINCT c.country) AS total_paises FROM transaction AS...	1 row(s) returned	0.297 sec / 0.000 sec

Con la consulta anterior, contamos los países sin repetir duplicados.

- Identifica la compañía amb la mitjana més gran de vendes.

```

22
23 • SELECT c.company_name, ROUND(AVG(t.amount), 2) AS media_ventas
24 FROM transaction AS t
25 JOIN company AS c
26 ON t.company_id = c.id
27 WHERE t.declined = 0
28 GROUP BY t.company_id
29 ORDER BY media_ventas DESC
30 LIMIT 1;

```

company_name	media_ventas
Ac Fermentum Incorporated	284.91

Result 6 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100115	12:42:24	SELECT COUNT(DISTINCT c.country) AS total_paises FROM transaction AS...	1 row(s) returned	0.297 sec / 0.000 sec
100116	12:53:32	SELECT c.company_name, ROUND(AVG(t.amount), 2) AS media_ventas FR...	1 row(s) returned	0.344 sec / 0.000 sec

Seleccionamos nombre de compañía y la media del precio (redondeada a dos decimales) por compañía, manteniendo el filtro de solo ventas (*declined* = 0), y ordenamos de mayor a menor por el valor de la media del precio, limitando solo al primer registro.

Exercici 3

Utilitzant només subconsultes (sense utilitzar JOIN):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

```

36 • SELECT *
37 FROM transaction AS t
38 WHERE declined = 0
39 AND EXISTS (
40     SELECT 1
41     FROM company AS c
42     WHERE c.id = t.company_id
43     AND c.country = "Germany"
44 );

```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8D877F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 7 x Apply

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100116	12:53:32	SELECT c.company_name, ROUND(AVG(t.amount), 2) AS media_ventas FR...	1 row(s) returned	0.344 sec / 0.000 sec
100117	13:01:35	SELECT * FROM transaction AS t WHERE declined = 0 AND EXISTS (SEL...	1000 row(s) returned	0.015 sec / 0.000 sec

Filtramos a través de una subconsulta las empresas alemanas, y filtramos de nuevo las transacciones exitosas con *declined* = 0.

- Lista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

The screenshot shows a SQL IDE window titled "Query 1" and "SQL File 4*". The query editor contains the following SQL code:

```
48 SELECT (  
49     SELECT company_name  
50     FROM company  
51     WHERE id = transaction.company_id  
52 ) AS nombre_compania,  
53 ROUND(AVG(amount), 2) AS media_empresa  
54 FROM transaction  
55 WHERE declined = 0  
56 GROUP BY company_id  
57 HAVING media_empresa > (  
58     SELECT AVG(amount)  
59     FROM transaction  
60     WHERE declined = 0)  
61 ORDER BY media_empresa DESC;
```

Below the query editor, the "Result Grid" shows the results of the query. It has two columns: "nombre_compania" and "media_empresa". The results are as follows:

nombre_compania	media_empresa
Ac Fermentum Incorporated	284.91
Pretium Neque Corp.	275.58
Urna Convallis Associates	273.57
At Associates	272.74
Metus Vitae Associates	270.05

The "Output" pane at the bottom shows the execution log. It includes the following entries:

#	Time	Action	Message	Duration / Fetch
100117	13:01:35	SELECT * FROM transaction AS t WHERE declined = 0 AND EXISTS (SEL...	1000 row(s) returned	0.015 sec / 0.000 sec
100118	13:05:46	SELECT (SELECT company_name FROM company WHERE id = trans...	48 row(s) returned	0.250 sec / 0.000 sec

Seleccionamos todas las empresas de la tabla **company** a través de una subconsulta en el **SELECT**, y lo nombramos *nombre_compania*. Agrupamos por *company_id* y sacaremos la media redondeada por compañía como *media_empresa*. Seguimos filtrando solo aquellas transacciones que no hayan sido rechazadas (*declined* = 0). Finalmente, creamos una subconsulta para sacar el valor de la media de transacción de todas las empresas (todavía con *declined* = 0), y lo comparamos con *media_empresa*, quedándonos con aquellos resultados que sean mayores a la media de todas las empresas. Ordené por *media_empresa* para que el resultado de la query quedara mejor visualmente, pero no se pedía en el enunciado, y de hecho hace que la query sea ligeramente más lenta que sin el **ORDER BY**.

- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

The screenshot shows a SQL IDE window titled "Query 1" and "SQL File 4*". The query editor contains the following SQL code:

```
64
65 • SELECT *
66 FROM company AS c
67 WHERE NOT EXISTS (
68     SELECT t.company_id
69     FROM transaction AS t
70     WHERE t.company_id = c.id
71 );
72
```

Below the query editor is a toolbar with icons for saving, running, and other database operations. The "Result Grid" is visible, showing a table with columns: id, company_name, phone, email, country, website. The first row shows all NULL values.

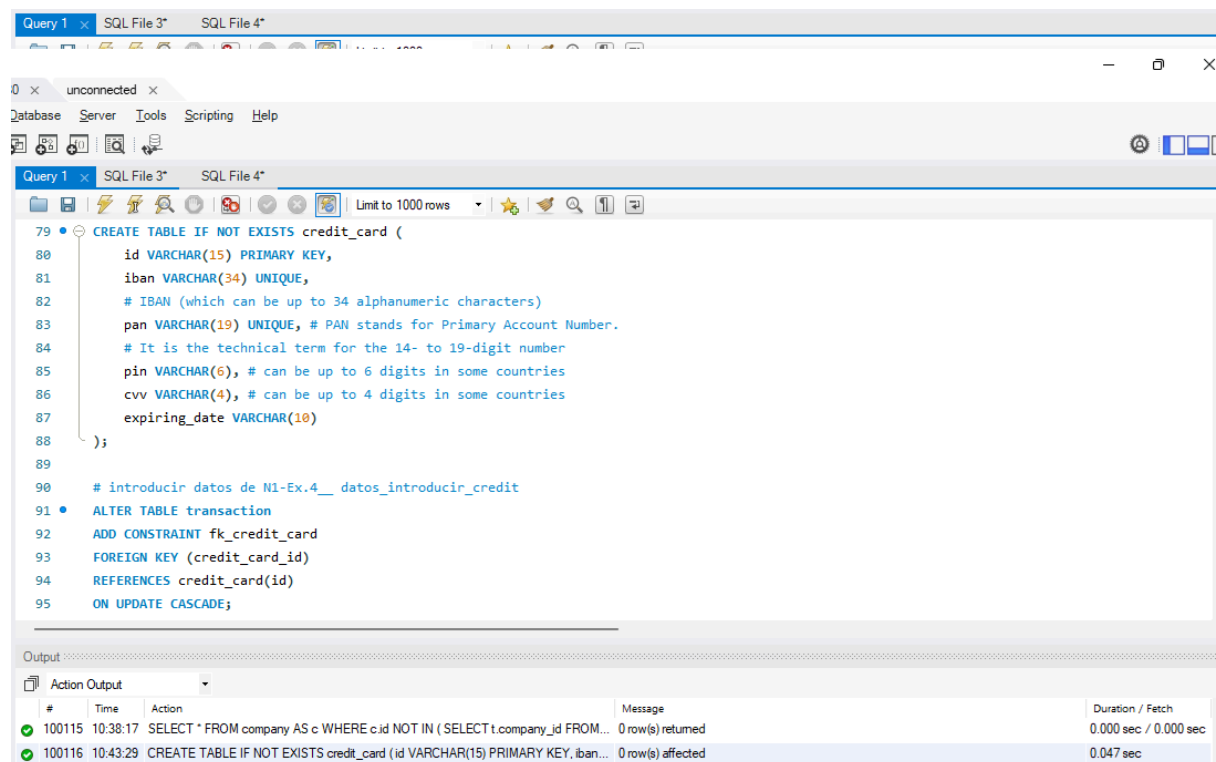
At the bottom, the "Output" pane shows the "Action Output" for the query execution:

#	Time	Action	Message	Duration / Fetch
100118	13:05:46	SELECT (SELECT company_name FROM company WHERE id = trans...	48 row(s) returned	0.250 sec / 0.000 sec
100119	13:11:16	SELECT * FROM company AS c WHERE NOT EXISTS (SELECT t.company...	0 row(s) returned	0.000 sec / 0.000 sec

Creemos una subconsulta con las compañías que aparecen en **transaction**. Esta "lista" la utilizamos en el **WHERE** para comparar con las compañías que aparecen en la tabla **company**. Al usar **WHERE NOT EXISTS**, el resultado deberían ser las empresas que aparecen en la tabla **company** que no existen en la tabla **transaction**.

Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.



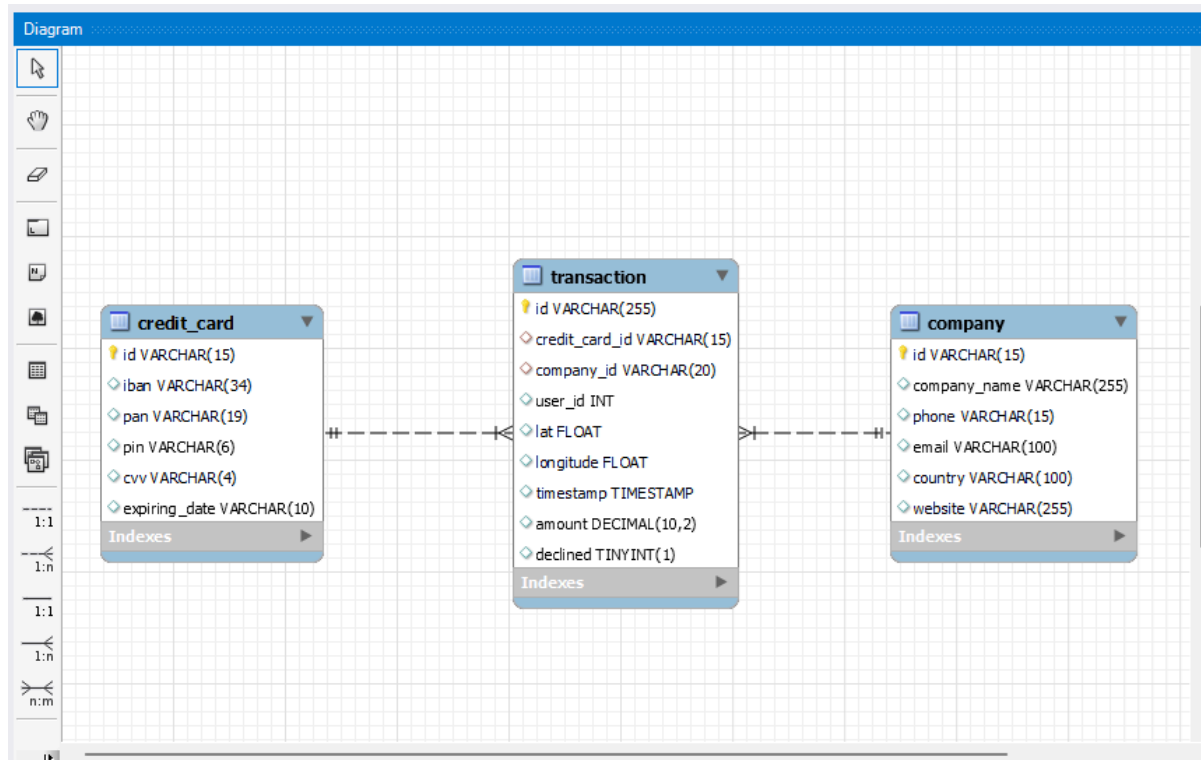
```
79 CREATE TABLE IF NOT EXISTS credit_card (  
80     id VARCHAR(15) PRIMARY KEY,  
81     iban VARCHAR(34) UNIQUE,  
82     # IBAN (which can be up to 34 alphanumeric characters)  
83     pan VARCHAR(19) UNIQUE, # PAN stands for Primary Account Number.  
84     # It is the technical term for the 14- to 19-digit number  
85     pin VARCHAR(6), # can be up to 6 digits in some countries  
86     cvv VARCHAR(4), # can be up to 4 digits in some countries  
87     expiring_date VARCHAR(10)  
88 );  
89  
90 # introducir datos de M1-Ex.4__ datos_introducir_credit  
91 ALTER TABLE transaction  
92 ADD CONSTRAINT fk_credit_card  
93 FOREIGN KEY (credit_card_id)  
94 REFERENCES credit_card(id)  
95 ON UPDATE CASCADE;
```

Output:

#	Time	Action	Message	Duration / Fetch
100115	10:38:17	SELECT * FROM company AS c WHERE c.id NOT IN (SELECT t.company_id FROM...	0 row(s) returned	0.000 sec / 0.000 sec
100116	10:43:29	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban...	0 row(s) affected	0.047 sec

Viendo la forma en que los datos están almacenados en **credit_card**, la tabla contiene un *id* como clave primaria que es un VARCHAR. Investigando los *IBAN* pueden ser de hasta un máximo de 34 caracteres y debería ser único. El *PAN* puede ser de hasta 19 caracteres. El *PIN* en algunos países es de hasta 6 números, y el *CVV* de hasta 4. La fecha de caducidad la hemos dejado como VARCHAR, ya que normalmente en las tarjetas se usa solo el mes y el año, y podremos extraer esa información con una función si lo necesitamos.

También alteramos la tabla **transaction** para que *credit_card_id* sea la clave foránea de **credit_card(id)** en la tabla **transaction**.



Hemos unido la nueva tabla con la tabla **transaction** uniendo el *id* de **credit_card** con el *credit_card_id* de **transaction**. Si quisiéramos ver la información de **credit_card** y **company** podemos usar la relación que ya existe a través de **transaction** como enlace de unión. La relación entre **credit_card** y **transaction** es de **1:N**, donde una *credit_card* puede tener varias transacciones, pero una transacción sólo puede estar asociada a una *credit_card*.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

The screenshot shows a SQL client window with a query editor and a results pane. The query editor contains the following SQL code:

```
1108 WHERE id = "CcU-2938";
1109
1110 • UPDATE credit_card
1111 SET iban = "TR323456312213576817699999"
1112 WHERE id = "CcU-2938";
1113
1114 • SELECT *
1115 FROM credit_card
1116 WHERE id = "CcU-2938";
1117
```

The results pane displays a table with the following data:

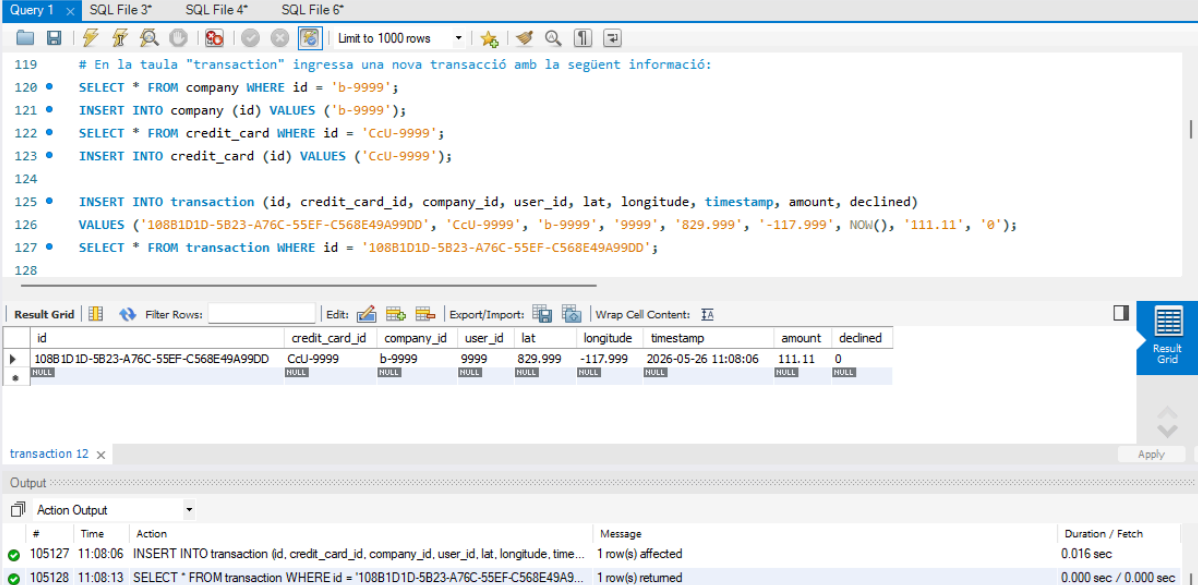
id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22
NULL	NULL	NULL	NULL	NULL	NULL

The output pane shows the execution of the query, indicating that 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0.

Actualizamos el valor de *iban* con el valor especificado en la tabla **credit_card** donde el valor de *id* coincide con el que nos proporcionan desde RRHH.

Exercici 6

En la taula "transaction" ingressa una nova transacció amb la següent informació:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and settings. The main query window contains the following SQL code:

```
119 # En la taula "transaction" ingressa una nova transacció amb la següent informació:
120 • SELECT * FROM company WHERE id = 'b-9999';
121 • INSERT INTO company (id) VALUES ('b-9999');
122 • SELECT * FROM credit_card WHERE id = 'CcU-9999';
123 • INSERT INTO credit_card (id) VALUES ('CcU-9999');
124
125 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)
126   VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', NOW(), '111.11', '0');
127 • SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
128
```

Below the query window is the 'Result Grid' showing the results of the queries. The first query returns one row with columns: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined. The second query returns one row with columns: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined. The third query returns one row with columns: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined.

The 'transaction 12' window shows the output of the third query, which is a single row with the following values: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined.

The 'Output' window shows the execution log. It contains two entries:

#	Time	Action	Message	Duration / Fetch
105127	11:08:06	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, time...	1 row(s) affected	0.016 sec
105128	11:08:13	SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A9...	1 row(s) returned	0.000 sec / 0.000 sec

Si intentamos introducir la información directamente en **transaction** nos dará un error debido a que no existe la **company** con **id** = 'b-9999' y tampoco **credit_card** con **id** = 'CcU-9999', por lo que habría que introducir esa información previamente para no romper las relaciones. No es muy buena práctica porque introducimos valores en **company** dejando muchos valores del resto de columnas vacíos. En un entorno laboral habría que preguntar qué hacer con respecto a estos casos, si introducirlos como hice yo, o si tenemos la información faltante de **company** y de **credit_card** y añadirla también, o si es mejor no hacer el **INSERT**.

Exercici 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL commands:

```
128
129 # Exercici 7
130 # Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.
131 • ALTER TABLE credit_card
132   DROP COLUMN pan;
133
134 • SELECT *
135   FROM credit_card;
136
137
```

The result grid displays the following data:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	5710	398	04/25/26

The output section shows the execution of the commands:

#	Time	Action	Message	Duration / Fetch
105129	11:12:07	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.187 sec
105130	11:12:14	SELECT * FROM credit_card LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.015 sec

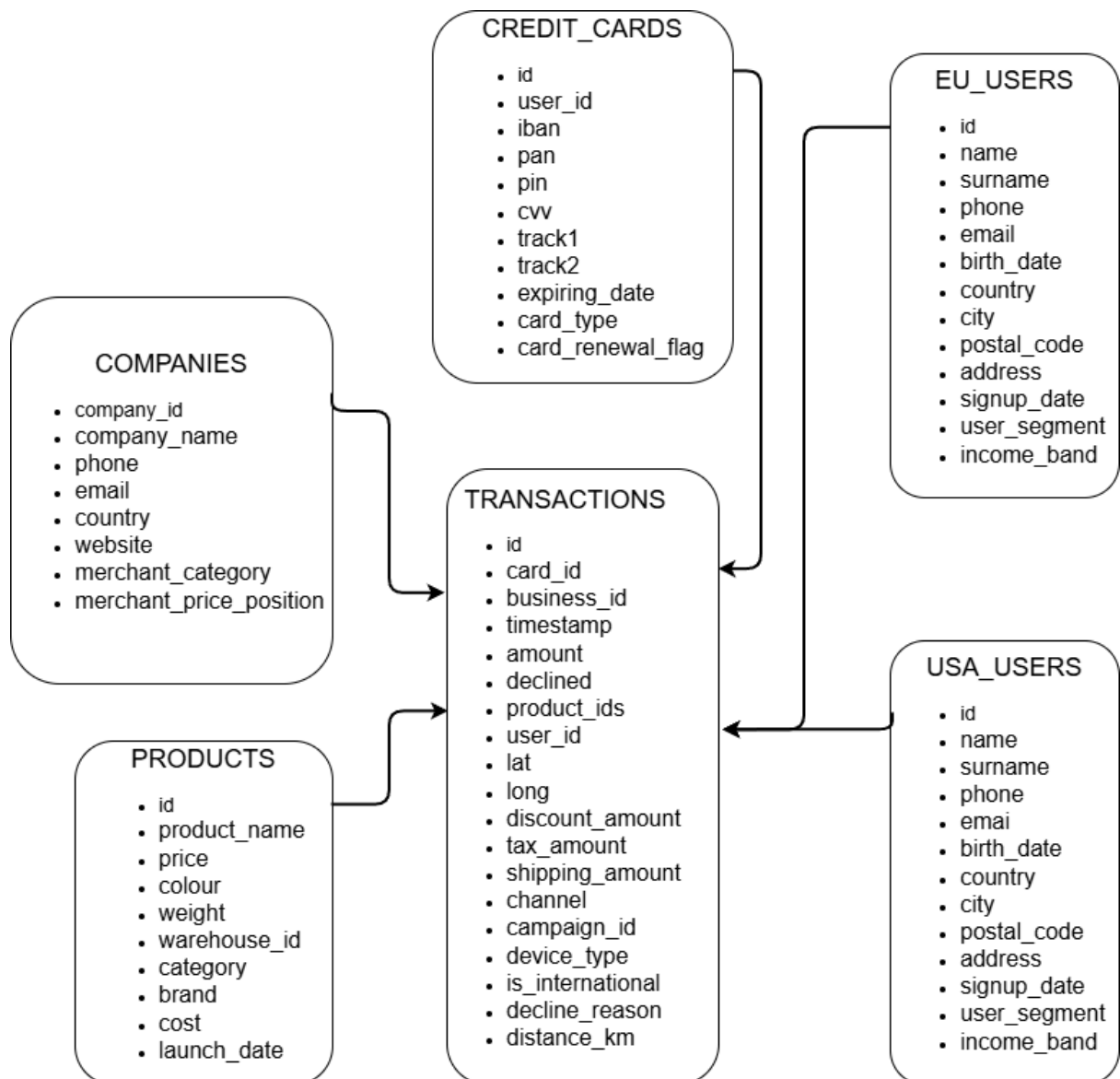
Eliminamos la columna *pan* de la tabla **credit_card**.

Exercici 8

Descarrega els arxius CSV que **trobaràs a l'apartat de recursos**:

- american_users.csv
- european_users.csv
- companies.csv
- credit_cards.csv
- transactions.csv

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:



Esquema conceptual antes de crear las tablas en SQL tras analizar y revisar los CSVs y su estructura y composición. Uniremos los CSVs con información de usuario en una sola tabla, pero estaría bien conservar la información del continente.

The screenshot shows a SQL query window with the following code:

```
1 # 8
2 # eliminamos la BBDD si existe y la creamos si no existe
3 DROP DATABASE IF EXISTS transactions_db;
4 CREATE DATABASE IF NOT EXISTS transactions_db;
5 USE transactions_db;
6
7 -- Creamos la tabla companies
8 CREATE TABLE IF NOT EXISTS companies (
9     company_id VARCHAR(10) PRIMARY KEY,
10    company_name VARCHAR(255),
11    phone VARCHAR(20),
12    email VARCHAR(100),
13    country VARCHAR(100),
14    website VARCHAR(255),
15    merchant_category VARCHAR(25),
16    merchant_price_position VARCHAR(25)
17 );
```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
105134	11:46:47	USE transactions_db	0 row(s) affected	0.000 sec
105135	11:46:56	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(10) PRIMARY ...	0 row(s) affected	0.063 sec

Eliminamos la nueva BBDD si existe y la creamos si no existe, y la usaremos para trabajar más cómodamente con ella. La BBDD la llamaremos **transactions_db**. También empezamos creando las tablas en función del análisis hecho de los CSV y el tipo de variables.

The screenshot shows a SQL query window with the following code:

```
19 -- Creamos la tabla credit_cards
20 CREATE TABLE IF NOT EXISTS credit_cards (
21     id VARCHAR(20) PRIMARY KEY,
22     user_id VARCHAR(15),
23     iban VARCHAR(34) NOT NULL UNIQUE,
24     # IBAN (which can be up to 34 alphanumeric characters)
25     pan VARCHAR(19) UNIQUE, # PAN stands for Primary Account Number.
26     # It is the technical term for the 14- to 19-digit number
27     pin VARCHAR(6), # can be up to 6 digits in some countries
28     cvv VARCHAR(4), # can be up to 4 digits in some countries
29     track1 VARCHAR(100),
30     track2 VARCHAR(100),
31     expiring_date DATE,
32     card_type VARCHAR(25),
33     card_renewal_flag BOOLEAN
34 );
```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
105135	11:46:56	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(10) PRIMARY ...	0 row(s) affected	0.063 sec
105136	11:49:05	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(20) PRIMARY KEY, ...	0 row(s) affected	0.047 sec

Basándonos en la investigación de ejercicios anteriores, creamos la tabla **credit_cards**.

```
Query 1  SQL File 4* x
Limit to 1000 rows

36 -- Creamos la tabla users donde juntaremos EU y USA_users
37 CREATE TABLE IF NOT EXISTS users (
38     id INT UNSIGNED PRIMARY KEY,
39     name VARCHAR(50),
40     surname VARCHAR(100),
41     phone VARCHAR(25),
42     email VARCHAR(200),
43     birth_date DATE,
44     continent VARCHAR(30),
45     country VARCHAR(100),
46     city VARCHAR(100),
47     postal_code VARCHAR(15),
48     address VARCHAR(200),
49     signup_date DATE,
50     user_segment VARCHAR(35),
51     income_band VARCHAR(25)
52 );
53
```

Output

#	Time	Action	Message	Duration / Fetch
14	18:03:16	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(20) PRIMARY ...	0 row(s) affected	0.046 sec
15	18:03:24	CREATE TABLE IF NOT EXISTS users (id INT UNSIGNED PRIMARY KEY, ...	0 row(s) affected	0.078 sec

Crearemos solo una tabla de usuarios para juntar en ella tanto los de usuarios de Europa como los de América, añadiendo una columna *continent* para que esa información no se pierda, ya que la estructura de las tablas es idéntica, y no hay duplicados en el *id* de cada usuario. Creamos el *id* como **INT** ya que son números y será más rápido y eficiente trabajar con ellos, y lo hacemos **UNSIGNED** porque solo usaremos números positivos. Para *birth_date* y *signup_date* tendremos que aplicar una función a la hora de introducir los datos y que no nos dé error y así poder operar con ellos en el futuro.

```
Query 1  SQL File 4* x  SQL File 6*
Limit to 1000 rows

52
53 -- Creamos la tabla transaction
54 CREATE TABLE IF NOT EXISTS transactions (
55     id VARCHAR(150) PRIMARY KEY,
56     card_id VARCHAR(20),
57     business_id VARCHAR(10),
58     timestamp TIMESTAMP,
59     amount DECIMAL(10, 2),
60     declined BOOLEAN,
61     product_ids VARCHAR(50),
62     user_id INT UNSIGNED,
63     lat FLOAT,
64     longitude FLOAT,
65     discount_amount DECIMAL(10, 2),
66     tax_amount DECIMAL(10, 2),
67     shipping_amount DECIMAL(10, 2),
68     channel VARCHAR(25),
69     campaign_id VARCHAR(25),

```

Output

#	Time	Action	Message	Duration / Fetch
105138	11:52:36	DROP TABLE 'transactions_db`.`users`	0 row(s) affected	0.015 sec
105139	11:52:44	CREATE TABLE IF NOT EXISTS users (id INT UNSIGNED PRIMARY KEY, name ...	0 row(s) affected	0.046 sec

Primera mitad de la tabla transactions. Mantenemos la misma estructura de datos entre las tablas y las claves foráneas. Todo lo que sean cantidades económicas lo introduciremos como decimal.

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL script for creating a table with various columns and foreign key constraints. The bottom pane shows the execution output, indicating that two tables were created successfully.

```
61 product_ids VARCHAR(50),
62 user_id INT UNSIGNED,
63 lat FLOAT,
64 longitude FLOAT,
65 discount_amount DECIMAL(10, 2),
66 tax_amount DECIMAL(10, 2),
67 shipping_amount DECIMAL(10, 2),
68 channel VARCHAR(25),
69 campaign_id VARCHAR(25),
70 device_type VARCHAR(20),
71 is_international BOOLEAN,
72 decline_reason VARCHAR(100),
73 distance_km FLOAT,
74 FOREIGN KEY (card_id) REFERENCES credit_cards(id),
75 FOREIGN KEY (business_id) REFERENCES companies(company_id),
76 FOREIGN KEY (user_id) REFERENCES users(id)
77 );
78
```

#	Time	Action	Message	Duration / Fetch
105139	11:52:44	CREATE TABLE IF NOT EXISTS users (id INT UNSIGNED PRIMARY KEY, name ...	0 row(s) affected	0.046 sec
105140	11:56:05	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(150) PRIMARY KEY, c...	0 row(s) affected	0.078 sec

La segunda mitad de la tabla transactions junto a las claves foráneas y a que tabla y columna hacen referencia.

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL script for loading data from CSV files into the 'companies' and 'credit_cards' tables. The bottom pane shows the execution output, indicating that the data was loaded successfully.

```
76 FOREIGN KEY (user_id) REFERENCES users(id)
77 );
78
79 # cargamos los CSVs
80
81 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'
82 INTO TABLE companies
83 FIELDS TERMINATED BY ','
84 ENCLOSED BY '"'
85 LINES TERMINATED BY '\n'
86 IGNORE 1 ROWS
87
88 (company_id, company_name, phone, email, country, website, merchant_category, merchant_price_position);
89
90 • SELECT * FROM companies;
91
92 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
93 INTO TABLE credit_cards
```

#	Time	Action	Message	Duration / Fetch
105140	11:56:05	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(150) PRIMARY KEY, c...	0 row(s) affected	0.078 sec
105141	11:57:51	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.047 sec

Cargamos los CSVs, habiéndolos puesto previamente en la carpeta Uploads de MySQL Server. Cargamos la información del CSV **companies** a la tabla de mismo nombre. Todas las tablas excepto la última (**transactions**) se suben con delimitador ','.

Query 1 SQL File 4* x SQL File 6*

```

88 (company_id, company_name, phone, email, country, website, merchant_category, merchant_price_position);
89
90 • SELECT * FROM companies;
91
92 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
93 INTO TABLE credit_cards
94 FIELDS TERMINATED BY ','
95 ENCLOSED BY '"'
96 LINES TERMINATED BY '\n'
97 IGNORE 1 ROWS
98
99 (id, user_id, iban, pan, pin, cvv, track1, track2, @expiring_date, card_type, card_renewal_flag)
100
101 SET expiring_date = STR_TO_DATE(@expiring_date, '%m/%d/%y');
102
103 • SELECT * FROM credit_cards;
104
105 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'

```

Output

#	Time	Action	Message	Duration / Fetch
105141	11:57:51	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.047 sec
105142	11:59:00	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.250 sec

Para el CSV de **credit_cards** aplicamos una función a *expiring_date* para convertir en **DATE** el string que viene en formato (mes/día/año).

Query 1 SQL File 4* x

```

102 SET expiring_date = STR_TO_DATE(@expiring_date, '%m/%d/%y');
103
104 • SELECT * FROM credit_cards;
105
106 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'
107 INTO TABLE users
108 FIELDS TERMINATED BY ','
109 ENCLOSED BY '"'
110 LINES TERMINATED BY '\n'
111 IGNORE 1 ROWS
112
113 (id, name, surname, phone, email, @birth_date, country, city, postal_code, address, signup_date, user_segment, income_band)
114
115 SET birth_date = STR_TO_DATE(@birth_date, '%b %d, %Y'),
116     continent = 'Europe';
117
118 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
119 INTO TABLE users

```

Output

#	Time	Action	Message	Duration / Fetch
18	18:08:12	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.250 sec
19	18:09:44	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.125 sec

Cargamos los datos de usuarios de Europa, transformando *birth_date* de un string a tipo fecha, y seteando *continent* como “Europe”.

Query 1 SQL File 4*

```

114
115 SET birth_date = STR_TO_DATE(@birth_date, '%b %d, %Y'),
116     continent = 'Europe';
117
118 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
119 INTO TABLE users
120 FIELDS TERMINATED BY ','
121 ENCLOSED BY '"'
122 LINES TERMINATED BY '\n'
123 IGNORE 1 ROWS
124
125 (id, name, surname, phone, email, @birth_date, country, city, postal_code, address, signup_date, user_segment, income_band)
126
127 SET birth_date = STR_TO_DATE(@birth_date, '%b %d, %Y'),
128     continent = 'America';
129
130 • SELECT * FROM users;
131
132 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'

```

Output

#	Time	Action	Message	Duration / Fetch
20	18:13:56	SELECT * FROM users LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
21	18:14:43	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.093 sec

Cargamos los datos de usuarios de América, transformando *birth_date* de un string a tipo fecha, y seteando *continent* como “America”.

Query 1 SQL File 4* SQL File 6*

```

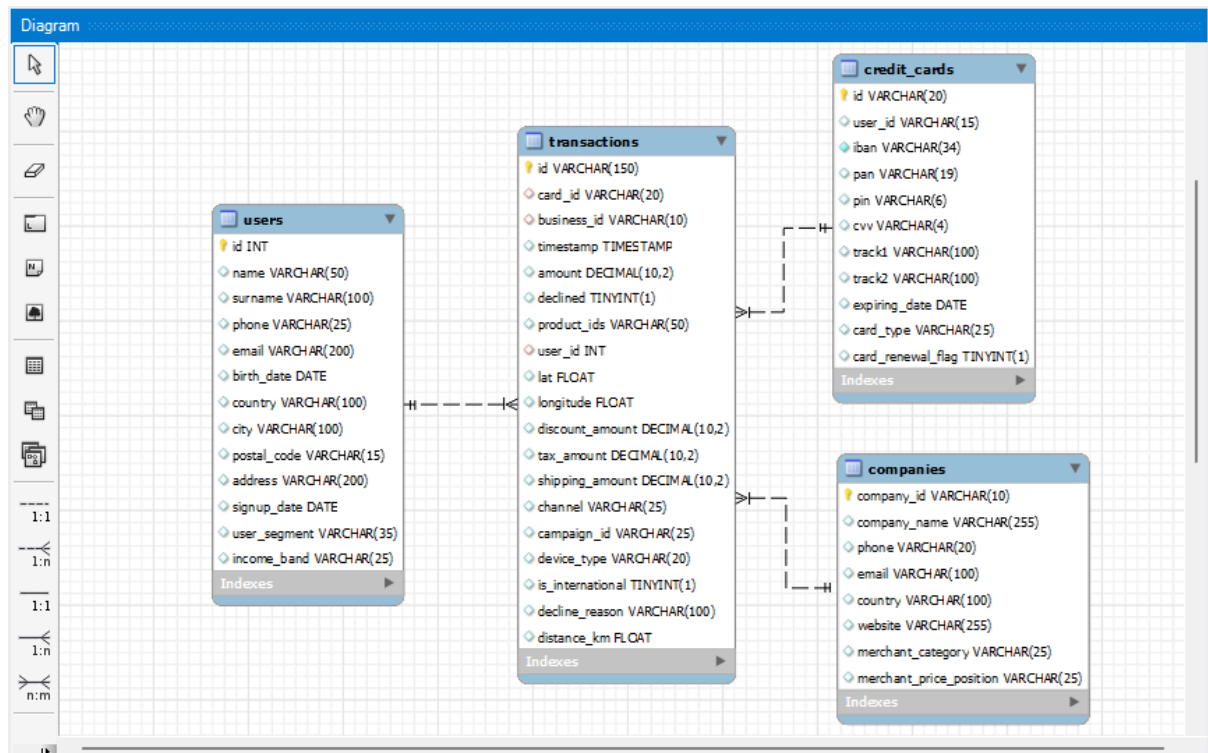
124
125 SET birth_date = STR_TO_DATE(@birth_date, '%b %d, %Y');
126
127 • SELECT * FROM users;
128
129 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
130 INTO TABLE transactions
131 FIELDS TERMINATED BY ','
132 ENCLOSED BY '"'
133 LINES TERMINATED BY '\n'
134 IGNORE 1 ROWS
135
136 (id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude, discount_amount, tax_amount,
137 shipping_amount, channel, campaign_id, device_type, is_international, decline_reason, distance_km);
138
139 • SELECT * FROM transactions;
140 # check de que las uniones funcionan bien
141 • SELECT *

```

Output

#	Time	Action	Message	Duration / Fetch
105144	12:01:24	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec
105145	12:02:34	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	4.656 sec

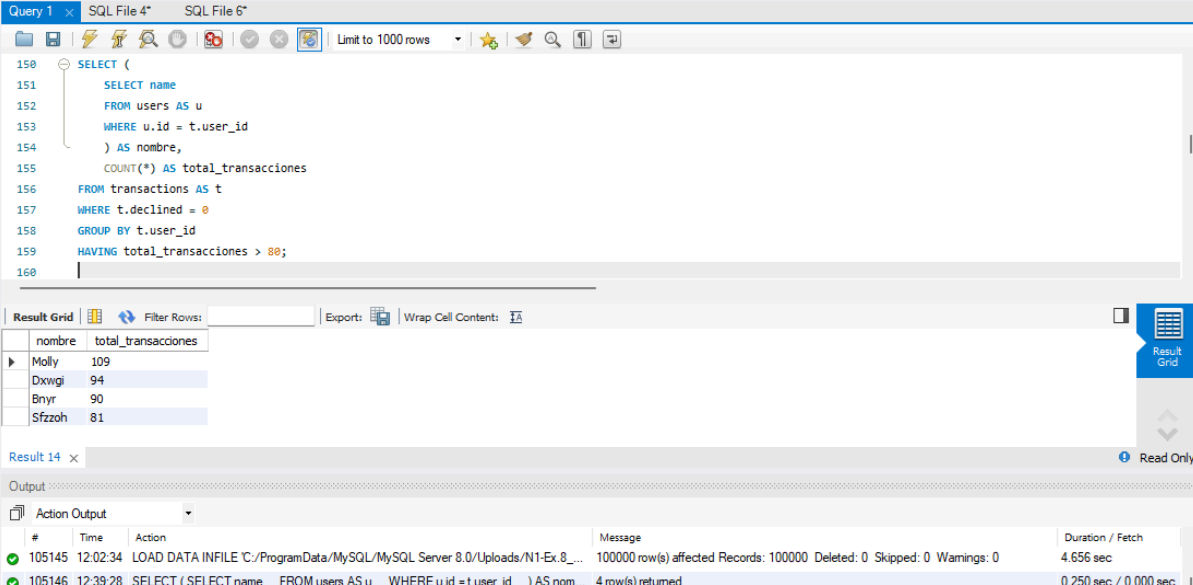
Cargamos los datos de **transactions**, esta vez usaremos como delimitador ‘,’ para separar la información entre columnas, ya que en la columna *product_ids* la información de los productos de cada transacción aparecen en forma de string separado por comas.



El esquema en forma de estrella deja como tabla central **transactions**, ya que es la única tabla de hechos; el resto de tablas son de dimensiones y rodean la tabla de hechos, ya que contienen información descriptiva de las métricas en **transactions**. Como mencionamos previamente, juntamos ambos CSVs de usuarios en una sola tabla **users**. Las relaciones que se establecen son de **1:N** con respecto a la tabla **transactions** (Un usuario puede tener varias transacciones pero una transacción solo puede pertenecer a un usuario, e igual para tarjetas de crédito y compañías)

Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.



The screenshot shows a MySQL IDE interface. The top pane contains a SQL query that uses a subquery to find users with more than 80 transactions. The bottom pane shows the results of the query in a table format.

```
150 SELECT (
151     SELECT name
152     FROM users AS u
153     WHERE u.id = t.user_id
154     ) AS nombre,
155     COUNT(*) AS total_transacciones
156 FROM transactions AS t
157 WHERE t.declined = 0
158 GROUP BY t.user_id
159 HAVING total_transacciones > 80;
160
```

nombre	total_transacciones
Molly	109
Dxwgi	94
Bnyr	90
Sfzzoh	81

Below the results, the 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
105145	12:02:34	LOAD DATA INFILE	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\N1-Ex.8_...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0 4.656 sec
105146	12:39:28	SELECT	(SELECT name FROM users AS u WHERE u.id = t.user_id) AS nom...	4 row(s) returned 0.250 sec / 0.000 sec

Utilizamos una subconsulta para seleccionar el nombre de los usuarios y poder unirlos con la tabla **transactions**. Contamos la cantidad de transacciones como *total_transacciones*, agrupando por *user_id*. Filtramos aquellas transacciones exitosas con *declined* = 0. Al querer saber los usuarios con más de 80 transacciones, y el **COUNT()** ser un método de agrupación, tendremos que filtrar usando un **HAVING**.

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query is as follows:

```
165 SELECT cc.iban, ROUND(AVG(t.amount), 2) AS media_gasto
166 FROM transactions AS t
167 JOIN companies AS c
168 ON t.business_id = c.company_id
169 JOIN credit_cards AS cc
170 ON t.card_id = cc.id
171 WHERE c.company_name = "Donec Ltd"
172 AND cc.card_type = "credit"
173 AND t.declined = 0
174 GROUP BY cc.iban
175 ORDER BY media_gasto DESC;
```

The results pane displays a table with two columns: **iban** and **media_gasto**. The data is as follows:

iban	media_gasto
US594988240619959562749388	638.82
US186466262266569948042760	629.76
US753450780359026774007480	608.25
CA255800760748246972054452	551.98
CA281689444843737671326853	546.90

The bottom pane shows the execution log with the following entries:

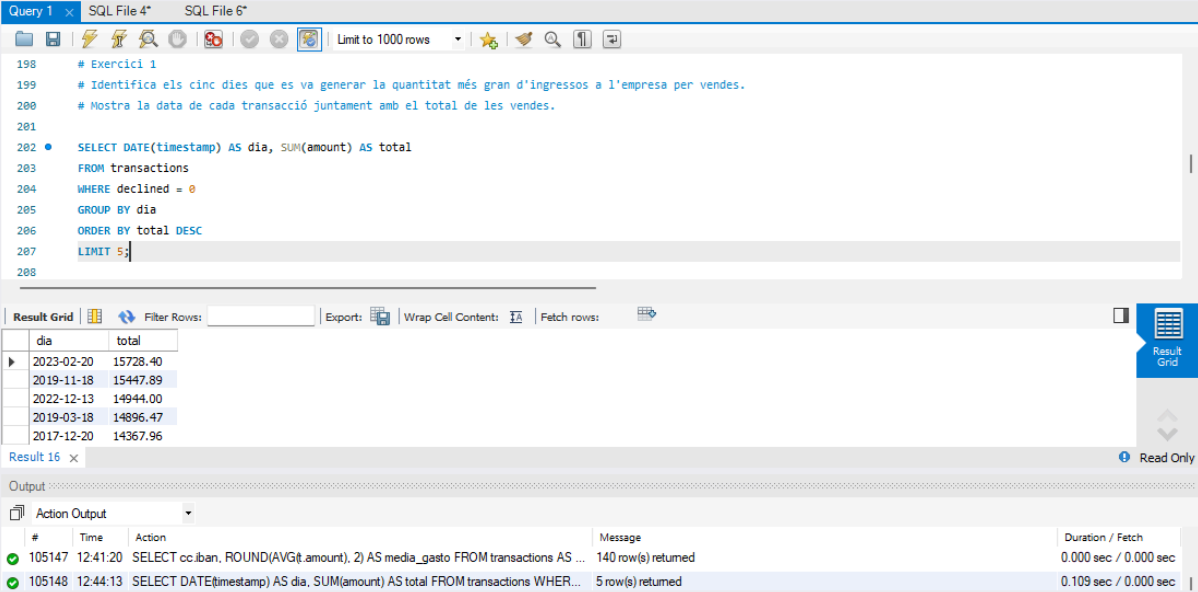
#	Time	Action	Message	Duration / Fetch
105146	12:39:28	SELECT (SELECT name FROM users AS u WHERE u.id = t.user_id) AS nom...	4 row(s) returned	0.250 sec / 0.000 sec
105147	12:41:20	SELECT cc.iban, ROUND(AVG(t.amount), 2) AS media_gasto FROM transactions AS ...	140 row(s) returned	0.000 sec / 0.000 sec

Unimos las tablas de **transactions**, **companies**, y **credit_cards** con un **JOIN**. Filtramos con un **WHERE** el nombre de la compañía que nos da el ejercicio, así como las transacciones exitosas con el *declined* = 0, así como las tarjetas de crédito con *card_type* = "credit" (existen 4 tipos distintos de *card_type*: "debit", "credit", "prepaid", y "premium_credit". Coloquialmente todas son tarjetas de crédito, pero estrictamente hablando, debit y prepaid no se consideran de crédito, y premium_credit sí que se consideraría una tarjeta de crédito, y sería interesante "haber hablado con el equipo de negocio" sobre si deberíamos considerar premium_credit, o las demás, para la consulta). Finalmente, agrupamos por *iban*, y seleccionamos para tanto *iban* como la media redondeada de cantidad por transacción por cada iban como *media_gasto*. Finalmente ordené de mayor a menor en función de la media de gasto, pero no es necesaria para responder el enunciado, y pierde eficiencia, pero a nivel estético queda mejor para presentarlo.

Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
198 # Exercici 1
199 # Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes.
200 # Mostra la data de cada transacció juntament amb el total de les vendes.
201
202 • SELECT DATE(timestamp) AS dia, SUM(amount) AS total
203 FROM transactions
204 WHERE declined = 0
205 GROUP BY dia
206 ORDER BY total DESC
207 LIMIT 5;
208
```

The results grid displays the following data:

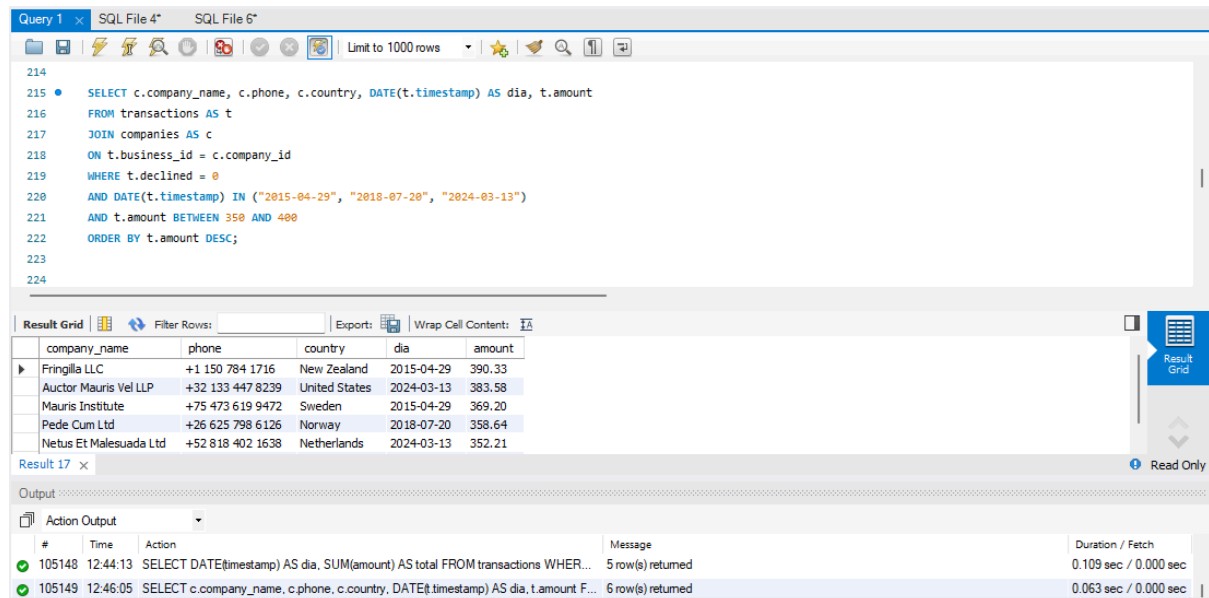
dia	total
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

The output pane shows the execution of the query, indicating that 5 rows were returned.

Utilizamos la función **DATE** para que extraiga la fecha de la columna *timestamp* como *dia*. Extraemos la suma de cantidad agrupada por *dia* como *total*, igualmente filtrando solo aquellas transacciones que han sido ventas con *declined* = 0, y ordenamos de mayor a menor en función del *total*, quedándonos sólo con los 5 primeros con un LIMIT.

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is as follows:

```
214
215 • SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS dia, t.amount
216 FROM transactions AS t
217 JOIN companies AS c
218 ON t.business_id = c.company_id
219 WHERE t.declined = 0
220 AND DATE(t.timestamp) IN ("2015-04-29", "2018-07-20", "2024-03-13")
221 AND t.amount BETWEEN 350 AND 400
222 ORDER BY t.amount DESC;
223
224
```

The results grid displays the following data:

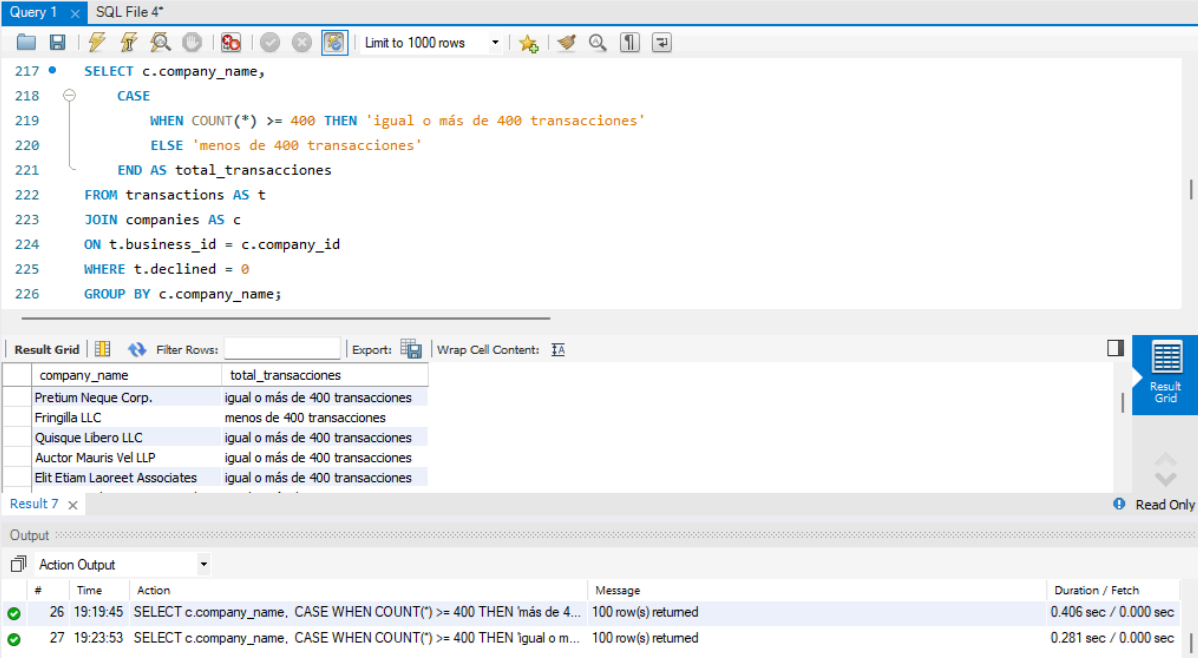
company_name	phone	country	dia	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21

The output pane shows the execution of the query, indicating that 6 row(s) were returned.

Seleccionamos los datos que nos pide el enunciado, uniendo las tablas **transactions** y **companies**. Aplicamos filtros a *amount* entre 350 y 400, *declined* = 0 para transacciones válidas, y con las 3 fechas que nos interesan. La manera más limpia es con un **IN**, y veo justificado ya que hacerlo con condicionales alargaría innecesariamente el código. También ordenamos de mayor a menor por *amount*.

Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.



The screenshot shows a SQL query in a file named 'SQL File 4*'. The query is as follows:

```
217 • SELECT c.company_name,  
218       CASE  
219         WHEN COUNT(*) >= 400 THEN 'igual o más de 400 transacciones'  
220         ELSE 'menos de 400 transacciones'  
221       END AS total_transacciones  
222 FROM transactions AS t  
223 JOIN companies AS c  
224   ON t.business_id = c.company_id  
225 WHERE t.declined = 0  
226 GROUP BY c.company_name;
```

The results are displayed in a table with two columns: **company_name** and **total_transacciones**.

company_name	total_transacciones
Pretium Neque Corp.	igual o más de 400 transacciones
Fringilla LLC	menos de 400 transacciones
Quisque Libero LLC	igual o más de 400 transacciones
Auctor Mauris Vel LLP	igual o más de 400 transacciones
Elit Etiam Laoreet Associates	igual o más de 400 transacciones

The bottom of the screenshot shows the 'Output' pane with two messages:

#	Time	Action	Message	Duration / Fetch
26	19:19:45	SELECT c.company_name, CASE WHEN COUNT(*) >= 400 THEN más de 4...	100 row(s) returned	0.406 sec / 0.000 sec
27	19:23:53	SELECT c.company_name, CASE WHEN COUNT(*) >= 400 THEN igual o m...	100 row(s) returned	0.281 sec / 0.000 sec

Seleccionamos el **company_name** de la tabla **companies**, y crearemos una nueva columna **total_transacciones** en base a si la suma de las transacciones exitosas (**WHERE declined = 0**) agrupadas por **company_name** tienen igual o más de 400 transacciones, o si tienen menos de 400 transacciones. Para ello lo hemos hecho con un condicional **CASE WHEN**.

Exercici 4

Elimina de la taula `transactions` el registre amb ID `000447FE-B650-4DCF-85DE-C7ED0EE1CAAD` de la base de dades.

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
245 FROM transactions
246 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
247
248 • DELETE FROM transactions
249 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
250
251 • SELECT *
252 FROM transactions
253 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
254
255 # Exercici 5
```

The results pane shows the output of the query, which is a table with 14 columns: `id`, `card_id`, `business_id`, `timestamp`, `amount`, `declined`, `product_ids`, `user_id`, `lat`, `longitude`, `discount_amount`, `tax_amount`, `shipping_amount`, `channel`, `campaign_id`, and `device_id`. The table is currently empty.

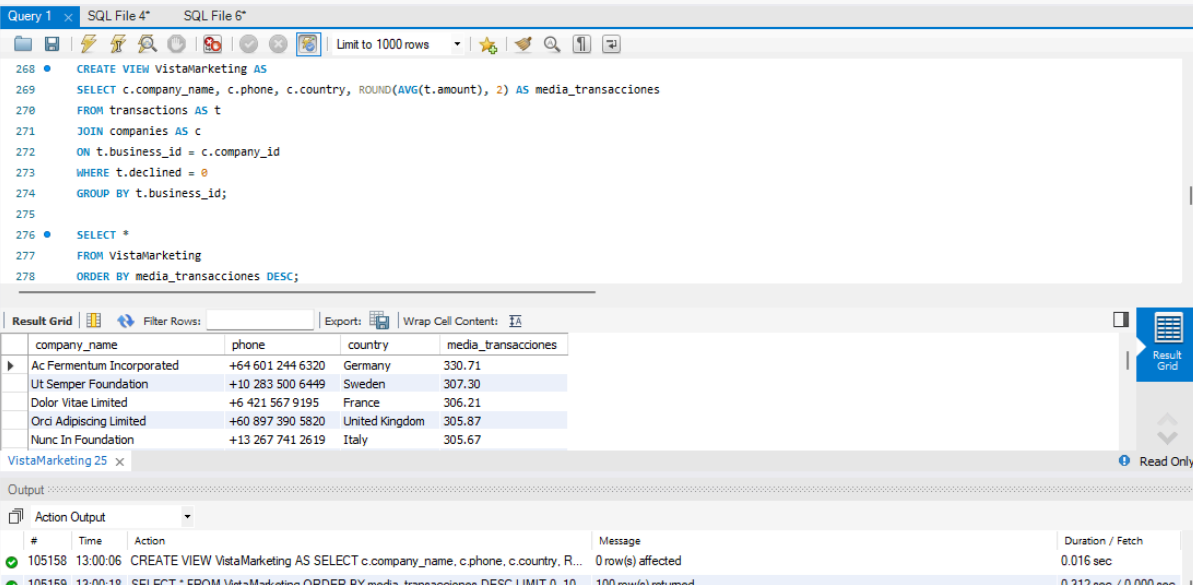
Below the table, the "Output" section shows the execution log:

#	Time	Action	Message	Duration / Fetch
105156	12:54:50	DELETE FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";	1 row(s) affected	0.031 sec
105157	12:54:56	SELECT * FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";	0 row(s) returned	0.000 sec / 0.000 sec

Eliminamos de la tabla **transactions** el registro cuyo *id* encaja con el del enunciado.

Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
268 • CREATE VIEW VistaMarketing AS
269 SELECT c.company_name, c.phone, c.country, ROUND(AVG(t.amount), 2) AS media_transacciones
270 FROM transactions AS t
271 JOIN companies AS c
272 ON t.business_id = c.company_id
273 WHERE t.declined = 0
274 GROUP BY t.business_id;
275
276 • SELECT *
277 FROM VistaMarketing
278 ORDER BY media_transacciones DESC;
```

The result grid displays the following data:

company_name	phone	country	media_transacciones
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.71
Ut Semper Foundation	+10 283 500 6449	Sweden	307.30
Dolor Vitae Limited	+6 421 567 9195	France	306.21
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.87
Nunc In Foundation	+13 267 741 2619	Italy	305.67

The output pane shows the execution of the SQL statements:

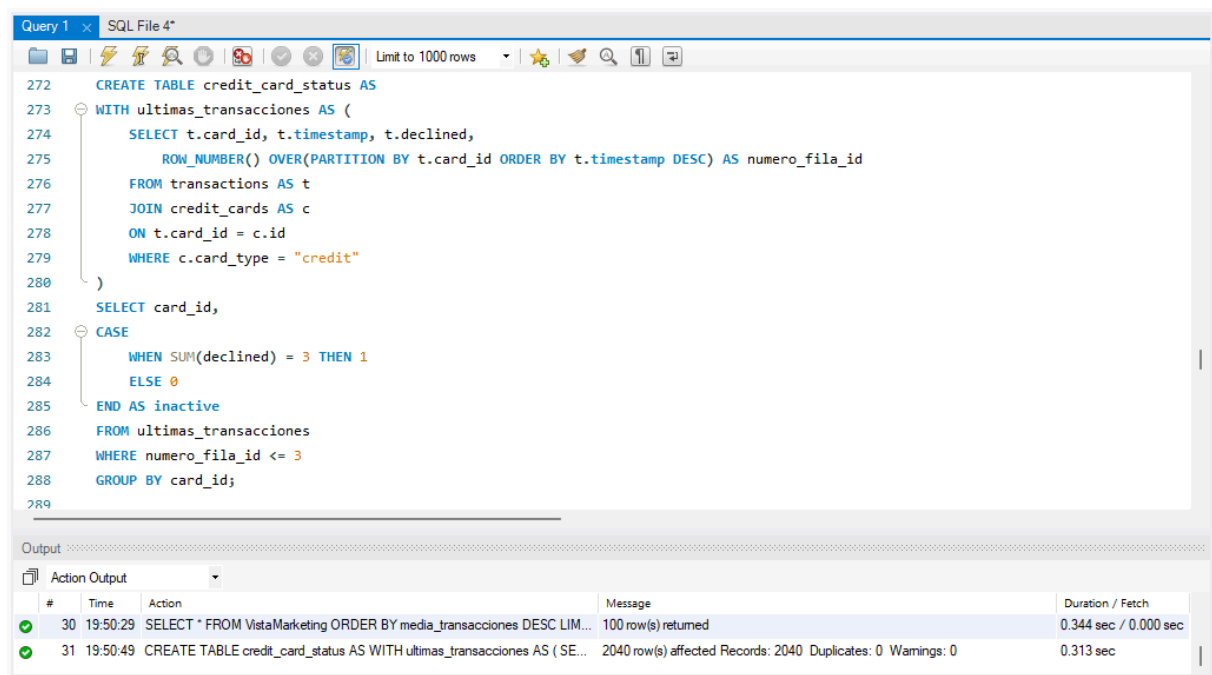
#	Time	Action	Message	Duration / Fetch
105158	13:00:06	CREATE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, R...	0 row(s) affected	0.016 sec
105159	13:00:18	SELECT * FROM VistaMarketing ORDER BY media_transacciones DESC LIMIT 0, 10...	100 row(s) returned	0.312 sec / 0.000 sec

Creemos una **VIEW** con la información que nos pide el enunciado. Seguimos aplicando el filtro para ver las compras (**WHERE declined = 0**). Sacamos la media de amount redondeada como *media_transacciones*, agrupada por *business_id*. Podríamos ordenar por *media_transacciones* para que fuera más visual, pero no es requerido en el enunciado.

Nivell 3

Exercici 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:



The screenshot shows a SQL query window titled "Query 1" with the following SQL code:

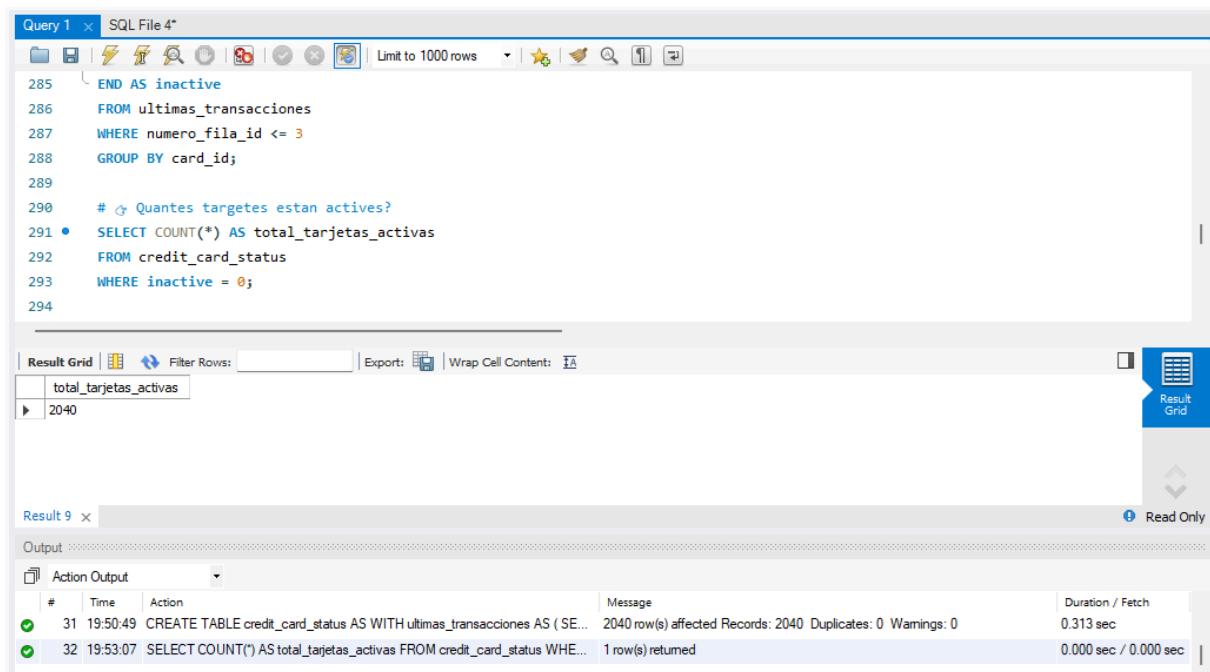
```
272 CREATE TABLE credit_card_status AS
273 WITH ultimas_transacciones AS (
274     SELECT t.card_id, t.timestamp, t.declined,
275            ROW_NUMBER() OVER(PARTITION BY t.card_id ORDER BY t.timestamp DESC) AS numero_fila_id
276     FROM transactions AS t
277     JOIN credit_cards AS c
278     ON t.card_id = c.id
279     WHERE c.card_type = "credit"
280 )
281 SELECT card_id,
282        CASE
283            WHEN SUM(declined) = 3 THEN 1
284            ELSE 0
285        END AS inactive
286 FROM ultimas_transacciones
287 WHERE numero_fila_id <= 3
288 GROUP BY card_id;
```

Below the query window, the "Output" pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
30	19:50:29	SELECT * FROM VistaMarketing ORDER BY media_transacciones DESC LIM...	100 row(s) returned	0.344 sec / 0.000 sec
31	19:50:49	CREATE TABLE credit_card_status AS WITH ultimas_transacciones AS (SE...	2040 row(s) affected Records: 2040 Duplicates: 0 Warnings: 0	0.313 sec

Creemos la nueva tabla **credit_card_status** a través de una tabla temporal **CTE ultimas_transacciones**. Esta **CTE** la creamos con la información de *card_id*, *timestamp*, y *declined*, y creamos la columna *numero_fila_id* con la función **ROW_NUMBER()**. Agrupar por *card_id* en **ROW_NUMBER()** hará que se asignen números a partir del 1, incrementando en 1 por cada nuevo registro del mismo *card_id*. Con cada *card_id* diferente, la cuenta vuelve a empezar desde 1. También ordenaremos descendentemente en función del *timestamp*, haciendo que las fechas más recientes obtengan números más bajos de **ROW_NUMBER()**. Como dice el enunciado, filtraremos solo por aquellas tarjetas cuyo *card_type* sea "credit" (como en un ejercicio anterior, quizás debiéramos incluir también las tarjetas *premium_credit*). Finalmente, filtraremos aquellos registros cuyo *numero_fila_id* sea igual o inferior a 3 para quedarnos con las 3 transacciones más recientes, y creando la columna *inactive*, le asignaremos un 1 (True) si en las últimas 3 transacciones *declined* suma 3, y un 0 (False) en cualquier otro caso, creando así la nueva tabla.

👉 ¿Cuántas tarjetas están activas?



The screenshot shows a SQL IDE window titled "Query 1" and "SQL File 4". The query editor contains the following SQL code:

```
285 END AS inactive
286 FROM ultimas_transacciones
287 WHERE numero_fila_id <= 3
288 GROUP BY card_id;
289
290 # ¿Cuántas tarjetas están activas?
291 • SELECT COUNT(*) AS total_tarjetas_activas
292 FROM credit_card_status
293 WHERE inactive = 0;
294
```

Below the query editor, the "Result Grid" shows a single row with the value 2040 for the column total_tarjetas_activas.

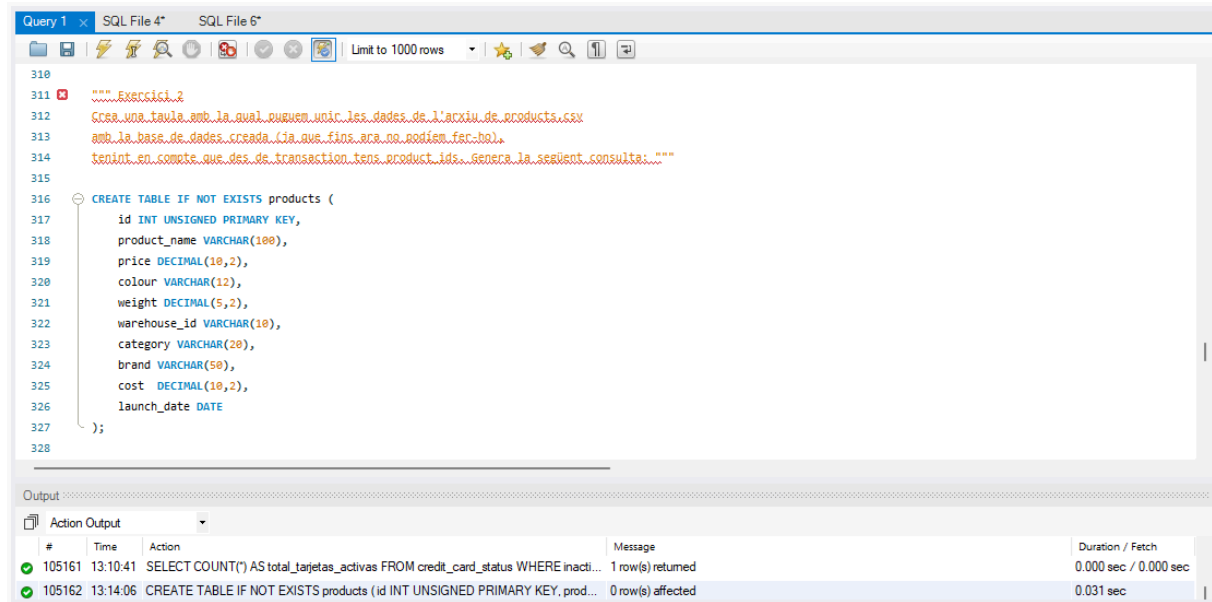
The "Output" pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
31	19:50:49	CREATE TABLE credit_card_status AS WITH ultimas_transacciones AS (SE...	2040 row(s) affected Records: 2040 Duplicates: 0 Warnings: 0	0.313 sec
32	19:53:07	SELECT COUNT(*) AS total_tarjetas_activas FROM credit_card_status WHE...	1 row(s) returned	0.000 sec / 0.000 sec

Con la tabla creada, para saber cuántas tarjetas están activas, sólo tenemos que contar los registros cuyo valor en *inactive* sea 0 (False), filtrando con un **WHERE**.

Exercici 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta:



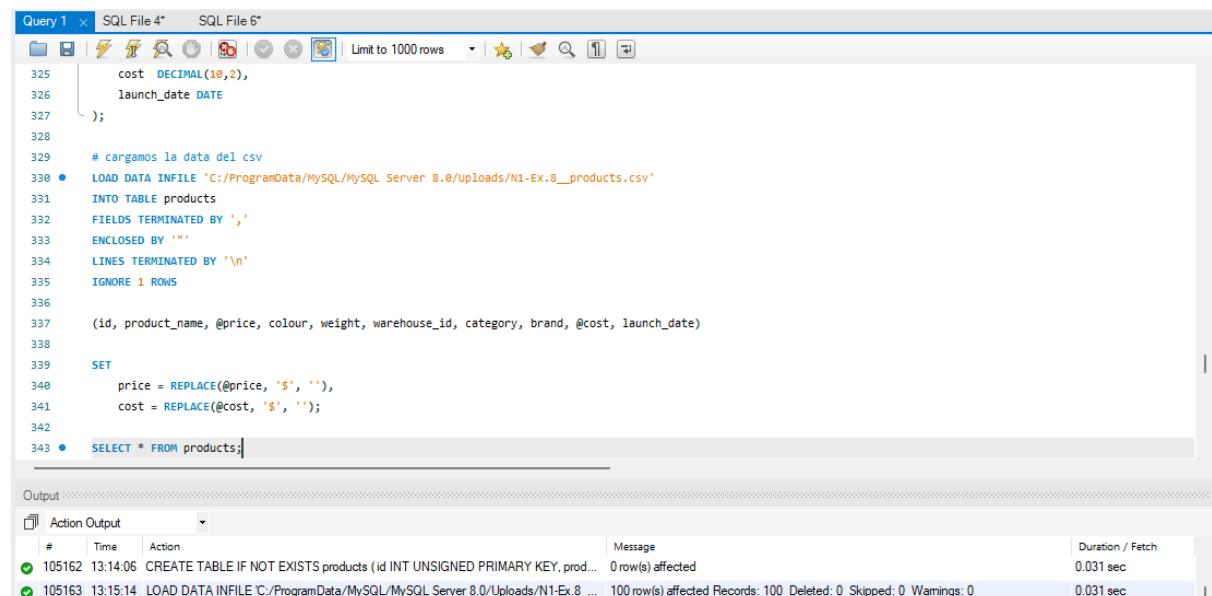
The screenshot shows a SQL IDE with a query editor and an output window. The query editor contains the following SQL code:

```
310
311 """ Exercici 2
312 Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv
313 amb la base de dades creada (ja que fins ara no podíem fer-ho).
314 tenint en compte que des de transaction tens product_ids. Genera la següent consulta: """
315
316 CREATE TABLE IF NOT EXISTS products (
317     id INT UNSIGNED PRIMARY KEY,
318     product_name VARCHAR(100),
319     price DECIMAL(10,2),
320     colour VARCHAR(12),
321     weight DECIMAL(5,2),
322     warehouse_id VARCHAR(10),
323     category VARCHAR(20),
324     brand VARCHAR(50),
325     cost DECIMAL(10,2),
326     launch_date DATE
327 );
328
```

The output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
105161	13:10:41	SELECT COUNT(*) AS total_tarjetas_activas FROM credit_card_status WHERE inacti...	1 row(s) returned	0.000 sec / 0.000 sec
105162	13:14:06	CREATE TABLE IF NOT EXISTS products (id INT UNSIGNED PRIMARY KEY, prod...	0 row(s) affected	0.031 sec

Creemos la tabla en función de los valores que tiene en el CSV y cómo sería interesante trabajarlos más adelante.



The screenshot shows a SQL IDE with a query editor and an output window. The query editor contains the following SQL code:

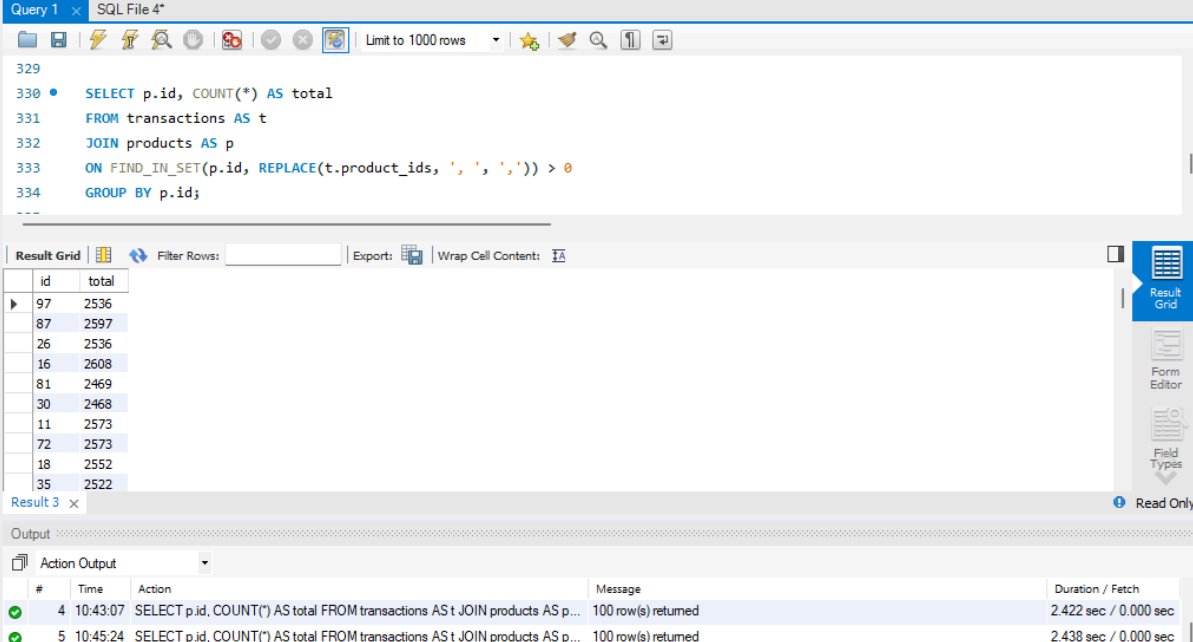
```
325     cost DECIMAL(10,2),
326     launch_date DATE
327 );
328
329 # cargamos la data del csv
330 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex_8_products.csv'
331 INTO TABLE products
332 FIELDS TERMINATED BY ','
333 ENCLOSED BY '"'
334 LINES TERMINATED BY '\n'
335 IGNORE 1 ROWS
336
337 (id, product_name, @price, colour, weight, warehouse_id, category, brand, @cost, launch_date)
338
339 SET
340 price = REPLACE(@price, '$', ''),
341 cost = REPLACE(@cost, '$', '');
342
343 SELECT * FROM products;
```

The output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
105162	13:14:06	CREATE TABLE IF NOT EXISTS products (id INT UNSIGNED PRIMARY KEY, prod...	0 row(s) affected	0.031 sec
105163	13:15:14	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex_8_...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.031 sec

Cargamos los datos del CSV **products**, y transformamos *price* y *cost*, quitándoles “\$” para poder introducirlo como **DECIMAL** y hacer operaciones en el futuro.

👉 Necesitamos conocer el número de veces que se ha vendido cada producto.



The screenshot shows a SQL IDE window titled "Query 1" and "SQL File 4*". The query editor contains the following SQL code:

```
329
330 • SELECT p.id, COUNT(*) AS total
331 FROM transactions AS t
332 JOIN products AS p
333 ON FIND_IN_SET(p.id, REPLACE(t.product_ids, ' ', ',')) > 0
334 GROUP BY p.id;
```

Below the query editor is the "Result Grid" showing the results of the query. The grid has two columns: "id" and "total". The results are as follows:

id	total
97	2536
87	2597
26	2536
16	2608
81	2469
30	2468
11	2573
72	2573
18	2552
35	2522

At the bottom of the IDE is the "Output" panel, which shows the execution log. It includes the following entries:

#	Time	Action	Message	Duration / Fetch
4	10:43:07	SELECT p.id, COUNT(*) AS total FROM transactions AS t JOIN products AS p...	100 row(s) returned	2.422 sec / 0.000 sec
5	10:45:24	SELECT p.id, COUNT(*) AS total FROM transactions AS t JOIN products AS p...	100 row(s) returned	2.438 sec / 0.000 sec

Para poder encontrar los diferentes **id** de **products** en la "lista" de **product_ids** de la tabla **transactions**, necesitamos primero eliminar los espacios entre comas con un **REPLACE(' ', ',')**. Ahora podemos aplicar correctamente la función **FIND_IN_SET()**, que buscará los diferentes **id** dentro de **product_ids** separados por comas. Agrupamos por **id** y aplicamos un **COUNT()** para saber cuántos registros hay por cada **id** diferente.